ITEC 116 – Systems Integration and Architecture

Enterprise System Lifecycles & Continuous Process Improvement

Topic: SDLC vs. ERP Lifecycle (ERPLC), Implementation Strategies, and Cutover Deployment Models

1. Systems Development Life Cycle (SDLC) vs. ERP Implementation Life Cycle (ERPLC)

While traditional custom software development follows a classic **System Development Life Cycle (SDLC)**, enterprise resource planning (ERP) and integration projects follow an **ERP Implementation Life Cycle (ERPLC)**.

The primary shift in thinking: **Custom SDLC focuses on determining requirements to build software from scratch, whereas ERPLC focuses on modifying business processes to fit prepackaged enterprise software.**

- **SDLC Goal:** Develop a custom system to support unique organizational requirements.
- **ERPLC Goal:** Implement a prepackaged enterprise platform to align business processes with industry best practices.
- **Cost of Customization:** Modifying core ERP software is expensive, increases ongoing vendor support fees, and complicates future system upgrades. Organizations should prioritize Business Process Re-engineering (BPR) over heavy code customization.

2. Implementation Plan Choices

When planning an enterprise system deployment, organizations select an implementation strategy based on budget, timeline, and risk tolerance:

- **Vanilla Strategy:** Uses out-of-the-box core ERP functionality and adapts business processes to match the software's built-in best practices. Minimizes costs, BPR overhead, and implementation time.
- **Middle-of-the-Road Strategy:** Involves moderate changes to core ERP modules alongside a significant amount of BPR. Balances customization needs with cost control.
- **Comprehensive Strategy:** The most expensive, lengthy, and high-risk approach. Involves deploying full software functionality, industry-specific modules, extensive BPR, and heavy customization of legacy systems.

3. Stages of the Traditional ERP Life Cycle

1. **Scope and Commitment Stage:** Establishes the business case, feasibility study, implementation scope, project team composition (internal SMEs, consultants, vendors), and top-management commitment.

2. **Analysis and Design Stage:** Conducts **Gap Analysis** (mapping differences between current workflows and system capabilities), designs change management plans, and creates a functional sandbox/prototype.
3. **Acquisition and Development Stage:** Purchases production licenses, builds the production infrastructure, configures security/roles, executes data migration mapping, and conducts preliminary end-user training.
4. **Implementation (Go-Live) Stage:** Converts operations from legacy environments to the new system, releases production platforms to end-users, and establishes help desk monitoring.
5. **Operation Stage:** Focuses on long-term knowledge transfer, ongoing end-user training, help desk handoffs, software contract management, and applying vendor patches/updates.

4. System Conversion (Cutover) Deployment Strategies

When transitioning from a legacy system to an integrated enterprise solution, organizations choose from four primary cutover methods:

- **Direct Cutover ("Big Bang"):** Immediate, direct transition to the new system. High risk, but lowest upfront cost and fastest deployment.
- **Parallel:** Runs both the legacy system and the new enterprise system concurrently for a set period. Highest upfront cost and workload, but lowest operational risk.
- **Phased:** Gradual, step-by-step migration of departments or functional modules over time. Minimizes operational disruption across the entire enterprise.
- **Pilot:** Deploys the complete system to a small target group or single location first to evaluate suitability before full-scale rollout.